

DASTOR Digital Field Manual

Digital edition compiled from the verified public chapter catalog and Execution Stack field guide on dastor.blocksifr.com for Instant Access purchasers.

Governing thesis: AI systems do not fail loudly. They fail quietly, and still act.

Generated: 2026-07-23T15:34:12.832Z

Source chapters: 40

Publisher: BlockSiFr LLC

Contents

1. AI Is Not Software (Part 1: The AI execution stack)
2. The AI Execution Stack Reference Model (Part 1: The AI execution stack)
3. Prompt Injection (Part 1: The AI execution stack)
4. Context Poisoning (Part 1: The AI execution stack)
5. Reasoning Failures and Goal Hijacking (Part 1: The AI execution stack)
6. Tool Exploitation (Part 1: The AI execution stack)
7. Execution Failures and Side Effects (Part 1: The AI execution stack)
8. Memory Is an Attack Surface (Part 1: The AI execution stack)
9. Indirect Prompt Injection in the Wild (Part 2: Input and context attacks)
10. Instruction Smuggling and Hidden Content (Part 2: Input and context attacks)
11. Retrieval Poisoning and Embedding Weaknesses (Part 2: Input and context attacks)
12. Cross-Tenant Leakage in RAG and Memory (Part 2: Input and context attacks)
13. Output Handling: When Text Becomes Code (Part 2: Input and context attacks)
14. Unbounded Consumption: Cost and Availability Attacks (Part 2: Input and context attacks)
15. The Planning Loop as an Attack Surface (Part 3: Planning, autonomy, and orchestration)
16. Tool Chaining and Capability Gravity (Part 3: Planning, autonomy, and orchestration)
17. Multi-Agent Handoffs and Delegation Abuse (Part 3: Planning, autonomy, and orchestration)

18. Agent Graph Poisoning and Workflow Attacks (Part 3: Planning, autonomy, and orchestration)
19. Human-in-the-Loop Failure Modes (Part 3: Planning, autonomy, and orchestration)
20. Observability Failures: When You Can't Reconstruct Why (Part 3: Planning, autonomy, and orchestration)
21. Sensitive Information Disclosure in Agentic Systems (Part 4: Data boundaries, privacy, and compliance)
22. Privacy Attacks: Inversion, Membership, and Regurgitation (Part 4: Data boundaries, privacy, and compliance)
23. Data Residency, Retention, and Right to Forget (Part 4: Data boundaries, privacy, and compliance)
24. Auditability at Scale: Traces, Logs, and Evidence (Part 4: Data boundaries, privacy, and compliance)
25. Compliance Mapping: SOC 2, ISO 27001, HIPAA, PCI DSS, EU AI Act (Part 4: Data boundaries, privacy, and compliance)
26. Incident Response for Agents: Containment and Rollback (Part 4: Data boundaries, privacy, and compliance)
27. Training and Fine-Tuning Poisoning (Part 5: Model, tool, and supply chain security)
28. Backdoors, Trojan Triggers, and Model Integrity (Part 5: Model, tool, and supply chain security)
29. Model Theft and Behavioral Cloning (Part 5: Model, tool, and supply chain security)
30. Tool Marketplace and Connector Supply Chain (Part 5: Model, tool, and supply chain security)
31. Runtime Compromise: Sandboxes, Scripts, Container Escapes (Part 5: Model, tool, and supply chain security)
32. CI/CD and Agent Builds in Production (Part 5: Model, tool, and supply chain security)
33. Deterministic Execution Gates (Part 6: Countermeasures and safe design patterns)
34. Tool Security Engineering Patterns (Part 6: Countermeasures and safe design patterns)
35. Retrieval Security Engineering Patterns (Part 6: Countermeasures and safe design patterns)
36. Memory Security Engineering Patterns (Part 6: Countermeasures and safe design patterns)
37. Evaluation, Red Teaming, and Regression Harnesses (Part 6: Countermeasures and safe design patterns)

38. Governance Operating Model and Change Control (Part 6: Countermeasures and safe design patterns)

39. The DASTOR Evidence Standard (ExecutionReceipt) (Part 6: Countermeasures and safe design patterns)

40. Building AI Systems That Fail Safely (Part 6: Countermeasures and safe design patterns)

A. Execution Stack — Attack Path and Countermeasure Field Guide

Chapter 1. AI Is Not Software

Part 1: The AI execution stack

Executive summary

Establishes why deterministic security assumptions fail when policy compliance is an inference problem under uncertainty.

Thesis

Establishes why deterministic security assumptions fail when policy compliance is an inference problem under uncertainty.

Scenario

A helpful enterprise assistant turns a plausible answer into an irreversible operational action.

Exploit paths

Ambiguity to wrong action

Ambiguity yields a plausible inference that becomes an irreversible operational action.

Long-context constraint loss

Constraints degrade under length and salience competition.

Automation bias

Operators accept plausible trajectories without verifying authority.

Root causes

Probabilistic instruction weighting

Policy compliance is inferred, not deterministically enforced.

Tool-calling expands blast radius

Language failure becomes system change.

Impacts

- **Financial:** Fraud, overpayment, or unauthorized transactions.
- **Operational:** Data corruption or unintended workflow completion.
- **Compliance:** Mishandled regulated data without a recoverable evidence trail.

Countermeasures

Execution discipline

- Gate irreversible actions with deterministic policy checks.
- Require traceable evidence for consequential execution.
- Separate inference from authority to act.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10, NIST AI RMF
 - Execution layers: input, inference, planning, execution
-

Chapter 2. The AI Execution Stack Reference Model

Part 1: The AI execution stack

Executive summary

Defines the canonical stack used throughout the manual and maps vendor agent abstractions to layers.

Thesis

Defines the canonical stack used throughout the manual and maps vendor agent abstractions to layers.

Scenario

The same workflow on multiple platforms exhibits the same failure modes because the stack is shared.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: input, tokenization, context-assembly, inference, planning, tool-selection, execution, memory
-

Chapter 3. Prompt Injection

Part 1: The AI execution stack

Executive summary

Prompt injection is input-level control-plane compromise; indirect injection weaponizes retrieval and tool outputs.

Thesis

Prompt injection is input-level control-plane compromise; indirect injection weaponizes retrieval and tool outputs.

Scenario

An agent summarizes a document; hidden instructions hijack output and tool use.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10
 - Execution layers: input, context-assembly
-

Chapter 4. Context Poisoning

Part 1: The AI execution stack

Executive summary

Retrieved equals trusted is the default failure mode; poisoning attacks target the retrieval layer.

Thesis

Retrieved equals trusted is the default failure mode; poisoning attacks target the retrieval layer.

Scenario

Internal knowledge-base ingestion pulls malicious content; the assistant repeats attacker-chosen answers.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10, NIST AI RMF
 - Execution layers: context-assembly
-

Chapter 5. Reasoning Failures and Goal Hijacking

Part 1: The AI execution stack

Executive summary

Planning is inference; attackers target objective inference and constraint weighting.

Thesis

Planning is inference; attackers target objective inference and constraint weighting.

Scenario

An agent tasked to help is reframed into unsafe or unauthorized subtasks.

Mapping

- Frameworks: OWASP LLM Top 10, NIST AI RMF
 - Execution layers: planning, inference
-

Chapter 6. Tool Exploitation

Part 1: The AI execution stack

Executive summary

Tools turn language failures into real-world side effects; tool selection and execution are exploitable surfaces.

Thesis

Tools turn language failures into real-world side effects; tool selection and execution are exploitable surfaces.

Scenario

An agent with file, database, or API tools is induced to call a tool with malicious but schema-valid arguments.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10
 - Execution layers: tool-selection, execution
-

Chapter 7. Execution Failures and Side Effects

Part 1: The AI execution stack

Executive summary

Blind execution, races, and missing confirmations convert inference errors into irreversible change.

Thesis

Blind execution, races, and missing confirmations convert inference errors into irreversible change.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution
-

Chapter 8. Memory Is an Attack Surface

Part 1: The AI execution stack

Executive summary

Persistent memory enables lasting injection, poisoning, and cross-session contamination.

Thesis

Persistent memory enables lasting injection, poisoning, and cross-session contamination.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: memory, context-assembly
-

Chapter 9. Indirect Prompt Injection in the Wild

Part 2: Input and context attacks

Executive summary

Modern IPI research and defenses for retrieval-mediated instruction hijack.

Thesis

Modern IPI research and defenses for retrieval-mediated instruction hijack.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10
 - Execution layers: input, context-assembly
-

Chapter 10. Instruction Smuggling and Hidden Content

Part 2: Input and context attacks

Executive summary

Display text is not parse text; HTML, CSS, and Unicode hide instructions.

Thesis

Display text is not parse text; HTML, CSS, and Unicode hide instructions.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: input, tokenization
-

Chapter 11. Retrieval Poisoning and Embedding Weaknesses

Part 2: Input and context attacks

Executive summary

Vector and ranking attacks undermine similarity-as-trust assumptions.

Thesis

Vector and ranking attacks undermine similarity-as-trust assumptions.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10
 - Execution layers: context-assembly
-

Chapter 12. Cross-Tenant Leakage in RAG and Memory

Part 2: Input and context attacks

Executive summary

Shared indexes and caches collapse isolation boundaries.

Thesis

Shared indexes and caches collapse isolation boundaries.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: context-assembly, memory
-

Chapter 13. Output Handling: When Text Becomes Code

Part 2: Input and context attacks

Executive summary

Model output treated as trusted code or query language creates RCE and injection paths.

Thesis

Model output treated as trusted code or query language creates RCE and injection paths.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: inference, execution
-

Chapter 14. Unbounded Consumption: Cost and Availability Attacks

Part 2: Input and context attacks

Executive summary

Loops, long contexts, and retries become economic and availability weapons.

Thesis

Loops, long contexts, and retries become economic and availability weapons.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: inference, planning
-

Chapter 15. The Planning Loop as an Attack Surface

Part 3: Planning, autonomy, and orchestration

Executive summary

Threat modeling the orchestration loop itself.

Thesis

Threat modeling the orchestration loop itself.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: planning
-

Chapter 16. Tool Chaining and Capability Gravity

Part 3: Planning, autonomy, and orchestration

Executive summary

Available tools pull agent behavior toward higher blast radius.

Thesis

Available tools pull agent behavior toward higher blast radius.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: tool-selection, execution
-

Chapter 17. Multi-Agent Handoffs and Delegation Abuse

Part 3: Planning, autonomy, and orchestration

Executive summary

Authenticated and unauthenticated handoffs create authority confusion.

Thesis

Authenticated and unauthenticated handoffs create authority confusion.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: planning, execution
-

Chapter 18. Agent Graph Poisoning and Workflow Attacks

Part 3: Planning, autonomy, and orchestration

Executive summary

Workflow graphs can be steered, substituted, or poisoned.

Thesis

Workflow graphs can be steered, substituted, or poisoned.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: planning
-

Chapter 19. Human-in-the-Loop Failure Modes

Part 3: Planning, autonomy, and orchestration

Executive summary

Rubber-stamping and incomplete context defeat approval theater.

Thesis

Rubber-stamping and incomplete context defeat approval theater.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: planning, execution
-

Chapter 20. Observability Failures: When You Can't Reconstruct Why

Part 3: Planning, autonomy, and orchestration

Executive summary

Missing spans mean you cannot prove what executed.

Thesis

Missing spans mean you cannot prove what executed.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution, memory
-

Chapter 21. Sensitive Information Disclosure in Agentic Systems

Part 4: Data boundaries, privacy, and compliance

Executive summary

Secrets and PII leak through answers, tools, and traces.

Thesis

Secrets and PII leak through answers, tools, and traces.

Mapping

- Frameworks: OWASP LLM Top 10, NIST AI RMF
 - Execution layers: inference, execution
-

Chapter 22. Privacy Attacks: Inversion, Membership, and Regurgitation

Part 4: Data boundaries, privacy, and compliance

Executive summary

Model and memory surfaces enable privacy attacks beyond classic appsec.

Thesis

Model and memory surfaces enable privacy attacks beyond classic appsec.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: inference, memory
-

Chapter 23. Data Residency, Retention, and Right to Forget

Part 4: Data boundaries, privacy, and compliance

Executive summary

Policy must bind memory, traces, and third-party tools.

Thesis

Policy must bind memory, traces, and third-party tools.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: memory
-

Chapter 24. Auditability at Scale: Traces, Logs, and Evidence

Part 4: Data boundaries, privacy, and compliance

Executive summary

Unify traces and logs into evidence suitable for auditors.

Thesis

Unify traces and logs into evidence suitable for auditors.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution
-

Chapter 25. Compliance Mapping: SOC 2, ISO 27001, HIPAA, PCI DSS, EU AI Act

Part 4: Data boundaries, privacy, and compliance

Executive summary

Map DASTOR controls and evidence to compliance frameworks.

Thesis

Map DASTOR controls and evidence to compliance frameworks.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution, memory
-

Chapter 26. Incident Response for Agents: Containment and Rollback

Part 4: Data boundaries, privacy, and compliance

Executive summary

Contain, freeze tools, and reverse when evidence allows.

Thesis

Contain, freeze tools, and reverse when evidence allows.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution
-

Chapter 27. Training and Fine-Tuning Poisoning

Part 5: Model, tool, and supply chain security

Executive summary

Poisoned data and adapters compromise model integrity.

Thesis

Poisoned data and adapters compromise model integrity.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10
 - Execution layers: inference
-

Chapter 28. Backdoors, Trojan Triggers, and Model Integrity

Part 5: Model, tool, and supply chain security

Executive summary

Signed artifacts and evaluation gates for model supply chain.

Thesis

Signed artifacts and evaluation gates for model supply chain.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: inference
-

Chapter 29. Model Theft and Behavioral Cloning

Part 5: Model, tool, and supply chain security

Executive summary

Abuse detection and rate limits for extraction.

Thesis

Abuse detection and rate limits for extraction.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: inference
-

Chapter 30. Tool Marketplace and Connector Supply Chain

Part 5: Model, tool, and supply chain security

Executive summary

Third-party tools and MCP servers require trust tiers.

Thesis

Third-party tools and MCP servers require trust tiers.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: tool-selection, execution
-

Chapter 31. Runtime Compromise: Sandboxes, Scripts, Container Escapes

Part 5: Model, tool, and supply chain security

Executive summary

Hardened runtimes and egress controls for code-executing agents.

Thesis

Hardened runtimes and egress controls for code-executing agents.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: execution
-

Chapter 32. CI/CD and Agent Builds in Production

Part 5: Model, tool, and supply chain security

Executive summary

Separate duties when agents can deploy.

Thesis

Separate duties when agents can deploy.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution
-

Chapter 33. Deterministic Execution Gates

Part 6: Countermeasures and safe design patterns

Executive summary

No side effects without a check that can be proven.

Thesis

No side effects without a check that can be proven.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution, planning
-

Chapter 34. Tool Security Engineering Patterns

Part 6: Countermeasures and safe design patterns

Executive summary

Allowlists, schemas, least privilege, and pre/post guardrails.

Thesis

Allowlists, schemas, least privilege, and pre/post guardrails.

Mapping

- Frameworks: OWASP LLM Top 10
 - Execution layers: tool-selection, execution
-

Chapter 35. Retrieval Security Engineering Patterns

Part 6: Countermeasures and safe design patterns

Executive summary

Provenance, sanitization, and poisoning detection.

Thesis

Provenance, sanitization, and poisoning detection.

Mapping

- Frameworks: MITRE ATLAS
 - Execution layers: context-assembly
-

Chapter 36. Memory Security Engineering Patterns

Part 6: Countermeasures and safe design patterns

Executive summary

Separate durable and working memory with approval for durable writes.

Thesis

Separate durable and working memory with approval for durable writes.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: memory
-

Chapter 37. Evaluation, Red Teaming, and Regression Harnesses

Part 6: Countermeasures and safe design patterns

Executive summary

Continuous regression against ATLAS and OWASP classes.

Thesis

Continuous regression against ATLAS and OWASP classes.

Mapping

- Frameworks: MITRE ATLAS, OWASP LLM Top 10, NIST AI RMF
 - Execution layers: inference, execution
-

Chapter 38. Governance Operating Model and Change Control

Part 6: Countermeasures and safe design patterns

Executive summary

Risk tolerance, change control, and operating cadence for AI systems.

Thesis

Risk tolerance, change control, and operating cadence for AI systems.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: planning, execution
-

Chapter 39. The DASTOR Evidence Standard (ExecutionReceipt)

Part 6: Countermeasures and safe design patterns

Executive summary

ExecutionReceipt as the evidence artifact for governed AI execution.

Thesis

ExecutionReceipt as the evidence artifact for governed AI execution.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution
-

Chapter 40. Building AI Systems That Fail Safely

Part 6: Countermeasures and safe design patterns

Executive summary

When in doubt, stop execution; degrade safely.

Thesis

When in doubt, stop execution; degrade safely.

Mapping

- Frameworks: NIST AI RMF
 - Execution layers: execution, planning, memory
-

Appendix A. Execution Stack — Attack Path and Countermeasure Field Guide

Compiled from the interactive Execution Stack on dastor.blocksifr.com. Inspect each layer in Normal Operation, Attack Path, and Countermeasures modes online.

Canonical stack:

```
Input → Tokenization → Context Assembly → Inference → Planning → Tool Selection →  
Execution → Memory  
└─ Future Context Assembly
```

01 Input

Accept user, document, web, email, API, and multimodal signals into the agent runtime before any model processing begins.

Trust boundaries

- Untrusted external content enters a control plane that may treat text as instruction
- Caller identity and source authenticity are asserted across systems
- Multimodal payloads cross modality-specific parsers with uneven controls

Attack surface

- Direct and indirect prompt injection
- Malicious documents and hidden instructions
- Multimodal payloads with steganographic or OCR-borne instructions
- Untrusted external content and social engineering
- Encoded or obfuscated instructions
- Identity and source ambiguity
- Data poisoning at ingestion

Failure modes

- Instruction/data boundary collapse
- Channel spoofing or forged provenance
- Scanner miss on obfuscated content
- Over-trust of authenticated but compromised senders
- Attachment parsers expanding attack surface

Propagation

- Injected instructions survive into tokenization and context assembly
- Primed goals skew inference and planning
- Tool selection and execution inherit attacker intent
- Poisoned content can be written into memory for lasting compromise

Consequences

- Policy bypass and unauthorized actions
- Unsafe tool priming
- Data exfiltration via later execution
- Compliance and safety violations
- Loss of audit integrity for downstream decisions

Countermeasures

****Prevent****

- Source provenance and channel authentication
- Instruction/data separation with explicit trust labels
- Content isolation and sandboxing for untrusted documents
- Input classification and sanitization
- Human confirmation for high-consequence requests

****Detect****

- Injection and hidden-instruction detectors
- Anomalous channel or identity signals
- Multimodal OCR/ stego inspection
- Rate and novelty alerts on inbound content

****Contain****

- Quarantine untrusted envelopes
- Strip or neutralize executable markup
- Downgrade privileges for tainted sessions

****Recover****

- Invalidate tainted sessions
- Reprocess from clean sources
- Revoke grants issued under tainted input

****Verify****

- Adversarial input test suites with pass/fail evidence
- Provenance records on accepted inputs

- Blocked-action evidence for rejected high-risk requests
- Alert-to-response timing for injection detections

Example scenario

- ****Initial condition:**** A support agent ingests a customer PDF attachment into an operations assistant.
 - ****Attack or failure:**** The PDF embeds white-on-white instructions to export CRM records and email them externally.
 - ****Propagation:**** Hidden text is tokenized, assembled into context, and influences planning and tool selection.
 - ****Consequence:**** Customer PII is exfiltrated through an authorized connector.
 - ****Countermeasure:**** Treat attachments as untrusted data; isolate content; require human confirmation before export tools.
 - ****Validation:**** Red-team PDF suite blocked; signed deny receipt for export tool; quarantine log with provenance.
-

02 Tokenization

Normalize and tokenize text (and related modalities) into the discrete units models and scanners consume.

Trust boundaries

- Security controls that inspect raw text may disagree with the model token view
- Normalization alters meaning before either path sees content

Attack surface

- Token boundary manipulation
- Unicode ambiguity and homoglyphs
- Invisible and control characters
- Encoding attacks and truncation
- Token-budget exhaustion
- Mismatch between raw-text scanners and token streams

Failure modes

- Scanner/model divergence
- Semantic drift after normalization
- Silent truncation of safety-critical clauses
- Log injection via control characters

Propagation

- Bypassed filters allow malicious instructions into context assembly
- Truncated policy text weakens later constraint enforcement
- Budget exhaustion degrades monitoring quality

Consequences

- Policy miss
- Undetected injection
- Availability and cost abuse
- Corrupted audit logs

Countermeasures

****Prevent****

- Unicode normalization and canonicalization
- Token-aware validation
- Length and budget limits
- Pinned tokenizer versions

****Detect****

- Raw-input vs token-stream comparison
- Hidden-character inspection
- Anomalous token density alerts

****Contain****

- Reject or quarantine divergent streams
- Hard fail on unsafe control characters

****Recover****

- Re-tokenize with known-good config
- Replay from raw stored input

****Verify****

- Differential tests: scanner vs model path
- Homoglyph and invisible-char fixtures
- Tokenizer version attestations

Example scenario

- ****Initial condition:**** A policy scanner blocks the phrase "ignore previous instructions" in UTF-8.
 - ****Attack or failure:**** An attacker encodes the same intent with homoglyphs and zero-width characters that the model still interprets.
 - ****Propagation:**** The token stream carries the instruction into context assembly while the scanner reports clean.
 - ****Consequence:**** Downstream planning treats attacker text as system instruction.
 - ****Countermeasure:**** Canonicalize before both scanner and model; compare raw and token views; fail closed on residual invisibles.
 - ****Validation:**** Fixture suite shows scanner and model agree; blocked samples produce deny evidence.
-

03 Context Assembly

Assemble system prompts, user content, retrieved knowledge, tool results, and memory into the working context window.

Trust boundaries

- Retrieved and tool-returned text must not inherit system trust
- Cross-tenant and cross-user data must not co-mingle
- Stale policy may silently weaken constraints

Attack surface

- Retrieval poisoning
- Cross-tenant contamination
- Stale policy and constraint loss
- Source-priority manipulation
- Context overflow and long-context dilution
- Untrusted memory retrieval
- System prompt leakage
- Context provenance failure

Failure modes

- Trust mixing
- Boundary collapse between system and untrusted text
- Overflow dropping critical constraints
- Over-weighting of low-confidence sources

Propagation

- Poisoned context steers inference
- Lost constraints enable unsafe planning and tool use
- Leaked system prompts aid further attacks

Consequences

- Steered answers and actions
- Cross-tenant leakage
- Hidden instructions executed later
- Compliance failures

Countermeasures

****Prevent****

- Trusted source hierarchy
- Context provenance tags
- Tenant isolation
- Policy pinning
- Context minimization
- Retrieval allowlists

****Detect****

- Provenance gaps
- Cross-tenant access anomalies
- Constraint-preservation tests failing
- Source-confidence outliers

****Contain****

- Drop or isolate untrusted segments
- Freeze retrieval for tainted indexes

****Recover****

- Rebuild context from pinned trusted sources
- Invalidate poisoned retrieval entries

****Verify****

- Constraint-preservation eval harness
- Tenant isolation tests
- Provenance completeness scores
- Freshness SLA evidence

Example scenario

- **Initial condition:** A RAG assistant retrieves top-k docs for a billing dispute.
 - **Attack or failure:** An attacker plants a high-similarity doc that elevates itself and embeds override instructions.
 - **Propagation:** Poisoned retrieval dominates context; inference and planning follow attacker goals.
 - **Consequence:** Unauthorized account credit is issued via tools.
 - **Countermeasure:** Retrieval allowlists, provenance scoring, pinned policies, and constraint-preservation tests.
 - **Validation:** Poisoned-doc red team blocked; retrieval audit shows source deny; signed decision for tool deny.
-

04 Inference

Generate candidate tokens, conclusions, and action proposals under model uncertainty.

Trust boundaries

- Model output is untrusted until validated
- Provider routing changes trust and residual risk
- Safety classifiers are themselves fallible controls

Attack surface

- Hallucination and misclassification
- Uncertainty concealment and overconfidence
- Adversarial examples
- Unsafe completion
- Model drift
- Sensitive-data reconstruction
- Model-routing risk
- Inference cost abuse

Failure modes

- Plausible but false fields
- Constraint loss in free-form text
- Silent routing to weaker models
- Safety classifier false negatives

Propagation

- False conclusions become plan premises
- Unsafe completions propose dangerous tools

- Leaked secrets enter later context or memory

Consequences

- Wrong operational decisions
- Secret leakage
- Unsafe plans
- Cost and availability impact

Countermeasures

****Prevent****

- Structured outputs and schemas
- Model routing policy
- Deterministic checks for consequential fields
- Cost and rate limits

****Detect****

- Confidence calibration monitors
- Safety classifiers
- Drift detection
- Secondary verification models

****Contain****

- Block free-form high-risk outputs
- Force human review for consequential conclusions

****Recover****

- Invalidate bad completions
- Re-run with pinned model and schema

****Verify****

- Eval and red-team scorecards
- Schema validation pass rates
- Human-review override logs
- Cost-limit enforcement evidence

Example scenario

- ****Initial condition:**** A clinical triage assistant proposes a next-step recommendation.
- ****Attack or failure:**** The model hallucinates a contraindicated medication with high confidence.

- ****Propagation:**** Planning encodes the drug into an order workflow; tools prepare a prescription action.
 - ****Consequence:**** Patient safety risk and liability.
 - ****Countermeasure:**** Structured output, formulary allowlist checks, secondary verification, human review for clinical conclusions.
 - ****Validation:**** Deterministic formulary reject; secondary model disagreement alert; HITL approval record.
-

05 Planning

Bind goals, decompose tasks, sequence steps, and estimate blast radius before tools are chosen.

Trust boundaries

- Plans must not expand authority beyond granted scope
- Intermediate steps can hide irreversible actions
- Trust-before-execution patterns apply to high-consequence plan commits

Attack surface

- Goal drift and deceptive decomposition
- Unsafe sequencing and hidden intermediate actions
- Excessive scope and blast-radius expansion
- Incorrect assumptions
- Irreversible planning
- Policy omission
- Cost-amplifying plans

Failure modes

- Autonomy overshoot
- Step explosion
- Missing HITL on protected actions
- Plans that cannot roll back

Propagation

- Unsafe plans force risky tool selection
- Hidden steps reach execution without scrutiny
- Budget blowouts degrade monitoring

Consequences

- Unauthorized subtasks
- Irreversible side effects
- Financial and operational damage
- Compliance breaches

Countermeasures

****Prevent****

- Goal binding
- Plan validation
- Consequence analysis
- Protected-action detection
- Step and budget limits
- Reversibility checks
- Human approval for high-consequence plans

****Detect****

- Plan anomaly detection
- Scope-expansion alerts
- Dependency inspection failures

****Contain****

- Freeze plan execution
- Strip non-approved steps

****Recover****

- Abort to last known-good plan
- Compensate completed reversible steps

****Verify****

- Plan review records
- Protected-action gate evidence
- Budget ceiling enforcement logs
- Approval receipts for escalated plans

Example scenario

- ****Initial condition:**** An infra agent is asked to "fix latency" on a production service.
- ****Attack or failure:**** Injected context expands the goal to rebuild clusters and rotate all credentials.

- ****Propagation:**** Tool selection pulls destructive cloud APIs; execution would mutate production.
 - ****Consequence:**** Outage and credential sprawl.
 - ****Countermeasure:**** Goal binding, blast-radius analysis, protected-action detection, and human approval before irreversible plans.
 - ****Validation:**** Plan rejected with STEP_UP evidence; no tool calls issued; signed plan-deny receipt.
-

o6 Tool Selection

Choose concrete tools and connectors from a catalog based on the validated plan and granted capabilities.

Trust boundaries

- Tool catalogs and descriptions are part of the attack surface
- Confused-deputy risk when tools act with broader authority
- Trust-before-execution applies before binding powerful tools

Attack surface

- Malicious or compromised tools
- MCP tool poisoning
- Tool substitution and namespace collision
- Overprivileged connectors
- Incorrect capability matching
- Parameter injection
- Shadow tools
- Unverified tool metadata
- Confused-deputy behavior

Failure modes

- Selecting a lookalike tool
- Trusting hostile descriptions
- Granting excess scopes
- Skipping schema validation

Propagation

- Wrong tool reaches execution with real credentials
- Hostile parameters become side effects
- Shadow tools bypass monitoring

Consequences

- Privilege expansion
- Data exfiltration
- Fraudulent transactions
- Supply-chain compromise

Countermeasures

****Prevent****

- Signed tool registry
- Capability allowlists
- Tool identity verification
- Least privilege
- Parameter schemas
- Destination restrictions
- Version pinning
- Capability-specific grants

****Detect****

- Registry integrity checks
- Unexpected tool selection alerts
- Metadata reputation signals

****Contain****

- Disable tainted tools
- Revoke grants
- Force re-selection from allowlist

****Recover****

- Roll back to pinned tool versions
- Rotate connector credentials

****Verify****

- Signature verification logs
- Allowlist enforcement evidence
- Schema validation failures recorded
- Grant scope attestations

Example scenario

- ****Initial condition:**** A developer agent needs to open a pull request.

- ****Attack or failure:**** A poisoned MCP tool description steers selection to a lookalike that exfiltrates tokens.
 - ****Propagation:**** Execution invokes the hostile connector with repo credentials.
 - ****Consequence:**** Source code and secrets leave the tenant.
 - ****Countermeasure:**** Signed registry, allowlists, identity verification, least privilege, pinned versions.
 - ****Validation:**** Unsigned tool rejected; selection audit shows allowlist miss; no credential use on deny.
-

07 Execution

Invoke tools and produce real-world side effects where language becomes consequence.

Trust boundaries

- Authority must be proven before side effects
- Fail-open behavior is a critical control defect
- Trust-before-execution and signed evidence apply at this layer

Attack surface

- Unauthorized API calls
- Destructive commands
- Code or infrastructure modification
- Data exfiltration
- Credential abuse
- Payment fraud
- Privilege escalation
- Rate abuse
- CI/CD compromise
- Irreversible external side effects
- Race conditions
- Fail-open behavior

Failure modes

- Blind execution without policy
- Sandbox escape
- Missing rollback
- Partial commits under failure

Propagation

- Successful malicious execution poisons memory and future context
- Stolen credentials enable further campaigns
- Irreversible changes permanently alter production

Consequences

- Data loss
- Fraud
- Production outage
- Legal and compliance exposure
- Safety incidents

Countermeasures

****Prevent****

- Pre-execution authorization
- Sandboxing
- Parameter enforcement
- Rate and transaction limits
- Step-up verification
- Human approval
- Fail-closed controls

****Detect****

- Anomalous side-effect monitoring
- Egress anomalies
- Race and replay detection

****Contain****

- Kill switches
- Immediate credential revocation
- Network egress cut

****Recover****

- Rollback and compensation
- Restore from known-good state
- Rotate secrets

****Verify****

- Signed execution receipts (identity, action, resource, decision, policy, trust/risk, time)
- Blocked-action evidence

- Rollback confirmation
- Alert-to-response timing

Example scenario

- ****Initial condition:**** A finance agent prepares a vendor payment under an approved plan.
 - ****Attack or failure:**** Parameter injection changes the destination account after approval.
 - ****Propagation:**** Without re-authorization, funds leave the organization; memory may record a false "paid" fact.
 - ****Consequence:**** Direct financial loss and audit failure.
 - ****Countermeasure:**** Bind approved parameters cryptographically; re-authorize mutations; fail closed; emit signed receipts.
 - ****Validation:**** Mutation triggers STEP_UP; deny receipt for changed IBAN; no payment posted.
-

o8 Memory

Persist short-term and long-term state that re-enters future context assembly — a feedback loop, not a terminal stage.

Trust boundaries

- Durable writes require higher trust than ephemeral turns
- Memory retrieval re-enters context assembly with lasting influence
- Trust-before-execution applies to memory writes and high-impact retrieval

Attack surface

- Memory poisoning
- False fact persistence
- Malicious instruction persistence
- Cross-user and cross-tenant contamination
- Sensitive-data retention
- Stale memory
- Unverifiable provenance
- Over-retention and incomplete deletion
- Embedding poisoning
- Future-context corruption

Failure modes

- Write without authorization
- Missing provenance

- Failed deletes leaving residues
- Retrieval of revoked facts

Propagation

- Poisoned memory re-enters context assembly on later sessions
- Stale or false facts steer inference and planning indefinitely
- Sensitive residues create compliance exposure across runs

Consequences

- Lasting compromise
- Repeated unsafe actions
- Privacy and residency violations
- Loss of organizational trust in the agent

Countermeasures

****Prevent****

- Memory-write authorization
- Provenance and source metadata
- Tenant and user isolation
- TTL and expiration
- Retrieval filtering
- Human review for durable high-impact facts

****Detect****

- Anomalous write patterns
- Provenance gaps
- Embedding drift / poison signals

****Contain****

- Memory quarantine
- Block retrieval of tainted classes

****Recover****

- Revocation and verified deletion
- Rebuild indexes from clean sources

****Verify****

- Deletion verification
- Provenance completeness

- Isolation tests
- Cryptographic integrity where appropriate
- Receipts for authorized durable writes

Example scenario

- ****Initial condition:**** An assistant stores "prefer vendor X" after a poisoned support thread.
 - ****Attack or failure:**** The false preference is written without provenance or approval.
 - ****Propagation:**** Future context assembly retrieves the preference; planning and tools favor vendor X repeatedly.
 - ****Consequence:**** Ongoing procurement bias and potential fraud.
 - ****Countermeasure:**** Authorize durable writes, attach provenance, quarantine low-trust facts, verify deletion on revoke.
 - ****Validation:**** Unauthorized write denied; quarantine record; post-delete retrieval returns empty with evidence.
-

Attribution

DASTOR is an AI security and countermeasures publication by BlockSiFr.